

Pipes & Redirection Cheat Sheet

Control where a command's input comes from and where its output and errors go.

The three streams

Number	Name	Default	Used for
0	stdin (standard input)	Keyboard	What the command reads
1	stdout (standard output)	Screen	Normal results
2	stderr (standard error)	Screen	Error and warning messages, kept apart so pipes don't carry them

Using it

Syntax	What it does	Syntax	What it does
cmd > file	Output to file (replaces it)	cmd 2> err.log	Errors to a file
cmd >> file	Output added to the end of file	cmd 2>> err.log	Errors added to the end of a file
cmd < file	Reads input from file	cmd > f 2>&1	Output and errors to one file
a b	Output of a becomes input of b	cmd &> file	Same thing, bash shorthand
a & b	Pipes output and errors (bash)	cmd 2> /dev/null	Throws errors away
cmd tee f	Screen and file (tee -a adds)	cmd &> /dev/null	Throws everything away

2>&1 means "send stream 2 to wherever stream 1 is going **right now**". **/dev/null** discards anything written to it.

Order matters (read left to right)

```
cmd > file 2>&1  ← 1 goes to file, then 2 copies 1 (file): both end up in file
cmd 2>&1 > file  ← 2 copies 1 (still the screen), then 1 moves to file: errors still on screen
sudo echo hi > /root/f  ← fails: your shell opens the file, not sudo
echo hi | sudo tee /root/f  ← works: tee runs as root and writes the file
```

Feeding text in

```
cat <<EOF > app.conf  ← here-doc: lines up to EOF become the input
host = $HOSTNAME  ← $variables and $(commands) are filled in
EOF  ← end marker must be alone on its line ('EOF' in quotes = no filling in)
grep -c x <<< "$var"  ← here-string: one string as input
diff <(sort a.txt) <(sort b.txt)  ← process substitution: a command's output used as a file
```

Reading the output: exit status

```
$ false | true; echo $?
0  ← $? of a pipe = status of the LAST command only; the failure is hidden
$ set -o pipefail; false | true; echo $?
1  ← pipefail: the pipe fails if any part fails
$ true | false | true; echo ${PIPESTATUS[@]}
0 1 0  ← bash: the status of every command in the last pipe
```

0 = success, anything else = failure. **a && b** runs b only if a succeeded; **a || b** runs b only if a failed.

Handy recipes

Command	Does
./backup.sh >> backup.log 2>&1	Logs a cron job's output and errors to one file
make 2>&1 tee build.log	Watches a build and saves a copy of everything
find / -name '*.conf' 2>/dev/null	Hides "Permission denied" noise
command -v nginx > /dev/null echo missing	Checks if a program is installed, silently
> app.log	Empties a file in place, without deleting it
set -o noclobber	Makes > refuse to overwrite a file (> forces it)
cmd 2>&1 grep -i error	Searches errors too (pipes normally carry only stdout)

Remember: > wipes the file first, >> adds to it. For "everything to one file", write > **file 2>&1** in that order.